

UC: Estruturas de Dados e Análise de Algoritmos

Título: Prática nº 02B – Atividade – EDA2B_Lista – Análise de Complexidade

Data: 29/04/2026

Integrantes:

- Sérgio Pinton Pavanelli - RA 123220202 (*líder: sergiopavanelli@gmail.com*)
- Samuel Zappala Batista - RA 12411504
- Elizabeth Stéphaney Guimarães Miranda - RA 123220604
- Gabriel Victor Dornelas Ferreira Sathler - 12319216
- Ana Luiza Mattos de Carvalho - RA 124114111
- Júlia Starling Negrini Fudoli - RA 124222027
- Ana Clara Domingos Dias Silva - RA 12316965
- Miguel Pedro Pinheiro - RA 12315515
- Geovana Dias de Almeida - RA 123221311

Tema: Estrutura de Dados e Análise de Algoritmos

Objetivo: Análise de Complexidade

Roteiro: Formar grupos e desenvolver as atividades propostas.

Entregas das atividades 1 a 3 solicitadas na lista de exercícios:

Atividade 1: Maior valor em um arranjo de n elementos não ordenados

Dado um arranjo A com n elementos não ordenados, o problema consiste em achar o **maior valor**. Como não há nenhuma propriedade de ordenação que possamos explorar, a única estratégia correta é varrer o arranjo uma única vez, mantendo uma variável **maior** que guarda o maior valor já visto e atualizando-a sempre que encontrarmos um elemento superior.

(a) Algoritmo ótimo

Pseudocódigo:

```
algoritmo AcharMaior(A, n)
    maior <- A[0]
    para i de 1 ate n - 1 faca
        se A[i] > maior entao
            maior <- A[i]
    retorne maior
```

C++:

```

#include <iostream>
using namespace std;

int acharMaior(int A[], int n) {
    int maior = A[0];
    for (int i = 1; i < n; i++) {
        if (A[i] > maior) {
            maior = A[i];
        }
    }
    return maior;
}

int main() {
    int n;
    cout << "Digite o tamanho do vetor: ";
    cin >> n;

    int A[100];
    cout << "Digite os " << n << " elementos do vetor A:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "A[" << i << "]: ";
        cin >> A[i];
    }

    cout << "\nMaior valor encontrado: " << acharMaior(A, n) << endl;
    return 0;
}

```

(b) Função de complexidade

O laço executa **exatamente $n - 1$ comparações**, independentemente do conteúdo do arranjo. A condição $A[i] > maior$ pode ou não atualizar **maior**, mas a comparação ocorre em todas as iterações.

- **Melhor caso:** $T(n) = n - 1 = \Theta(n)$. Exemplo: vetor em ordem decrescente — **maior** é inicializado com $A[0]$ (que já é o maior) e nenhuma atribuição extra é feita.
- **Pior caso:** $T(n) = n - 1 = \Theta(n)$. Exemplo: vetor em ordem crescente — a cada iteração **maior** é atualizado.
- **Caso médio:** $T(n) = \Theta(n)$. O número de comparações é fixo; apenas o número de atualizações varia.

Conclusão: o algoritmo é **ótimo**, pois qualquer algoritmo correto precisa inspecionar todos os n elementos pelo menos uma vez (limite inferior $\Omega(n)$), e este atinge $\Theta(n)$.

Atividade 2: Maior e menor valor em uma única passada

Mantemos duas variáveis, **maior** e **menor**, ambas inicializadas com $A[0]$. A cada elemento $A[i]$ realizamos as comparações necessárias para atualizar uma das duas variáveis.

(a) Algoritmo ótimo

Pseudocódigo:

```
algoritmo AcharMaiorMenor(A, n, maior, menor)
    maior ← A[0]
    menor ← A[0]
    para i de 1 ate n - 1 faca
        se A[i] > maior entao
            maior ← A[i]
        senao se A[i] < menor entao
            menor ← A[i]
    retorne maior, menor
```

C++:

```
#include <iostream>
using namespace std;

void acharMaiorMenor(int A[], int n, int &maior, int &menor) {
    maior = A[0];
    menor = A[0];
    for (int i = 1; i < n; i++) {
        if (A[i] > maior) {
            maior = A[i];
        } else if (A[i] < menor) {
            menor = A[i];
        }
    }
}

int main() {
    int n;
    cout << "Digite o tamanho do vetor: ";
    cin >> n;

    int A[100];
    cout << "Digite os " << n << " elementos do vetor A:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "A[" << i << "]: ";
        cin >> A[i];
    }

    int maior, menor;
    acharMaiorMenor(A, n, maior, menor);

    cout << "\nMaior valor: " << maior << endl;
    cout << "Menor valor: " << menor << endl;
```

```
    return 0;
}
```

(b) Função de complexidade

A cada iteração executa-se **pelo menos 1 e no máximo 2 comparações** entre o elemento $A[i]$ e as variáveis **maior**/**menor**.

- **Melhor caso:** $T(n) = n - 1 = \theta(n)$. Ocorre quando todo elemento satisfaz $A[i] > \text{maior}$ (vetor crescente) — apenas a primeira comparação é executada por iteração.
- **Pior caso:** $T(n) = 2(n - 1) = \theta(n)$. Ocorre quando nenhum elemento entra no primeiro **se**, forçando a segunda comparação em toda iteração (ex.: $A[0]$ é o maior e os demais decrescem).
- **Caso médio:** $T(n) = \theta(n)$, com constante entre 1 e 2.

Otimização (técnica dos pares): processando o vetor em pares — primeiro compara-se $A[i]$ com $A[i+1]$, depois compara-se o menor do par com **menor** e o maior do par com **maior** — obtêm-se aproximadamente $3\lfloor n/2 \rfloor - 2$ comparações, melhor constante mas mesma ordem $\theta(n)$.

Conclusão: o algoritmo é **ótimo em ordem assintótica**, com $T(n) = \theta(n)$.

Atividade 3: Maior e segundo maior valor

Mantemos duas variáveis: **max1** (maior) e **max2** (segundo maior). Inicializamos comparando os dois primeiros elementos. A cada novo elemento $A[i]$, se ele superar **max1**, o antigo **max1** desce para **max2**; senão, se superar **max2**, atualizamos apenas **max2**.

(a) Algoritmo ótimo

Pseudocódigo:

```
algoritmo AcharMaiorESegundoMaior(A, n, max1, max2)
    se n < 2 entao
        retorne erro
    se A[0] >= A[1] entao
        max1 <- A[0]
        max2 <- A[1]
    senao
        max1 <- A[1]
        max2 <- A[0]
    para i de 2 ate n - 1 faca
        se A[i] > max1 entao
            max2 <- max1
            max1 <- A[i]
        senao se A[i] > max2 entao
            max2 <- A[i]
    retorne max1, max2
```

C++:

```
#include <iostream>
using namespace std;

void acharMaior2(int A[], int n, int &max1, int &max2) {
    if (A[0] >= A[1]) {
        max1 = A[0];
        max2 = A[1];
    } else {
        max1 = A[1];
        max2 = A[0];
    }
    for (int i = 2; i < n; i++) {
        if (A[i] > max1) {
            max2 = max1;
            max1 = A[i];
        } else if (A[i] > max2) {
            max2 = A[i];
        }
    }
}

int main() {
    int n;
    cout << "Digite o tamanho do vetor (n >= 2): ";
    cin >> n;

    int A[100];
    cout << "Digite os " << n << " elementos do vetor A:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "A[" << i << "]: ";
        cin >> A[i];
    }

    int max1, max2;
    acharMaior2(A, n, max1, max2);

    cout << "\nMaior valor: " << max1 << endl;
    cout << "Segundo maior valor: " << max2 << endl;
    return 0;
}
```

(b) Função de complexidade

Após a inicialização (1 comparação para os dois primeiros elementos), o laço percorre $n - 2$ elementos, fazendo **1 ou 2 comparações por iteração**.

- **Melhor caso:** $T(n) = (n - 2) + 1 = n - 1 = \Theta(n)$. Ocorre quando cada $A[i]$ é maior que max1 (vetor crescente) — só a primeira comparação é executada por iteração.
- **Pior caso:** $T(n) = 2(n - 2) + 1 = 2n - 3 = \Theta(n)$. Ocorre quando nenhum elemento supera max1 , mas todos exigem o teste contra max2 .

- **Caso médio:** $T(n) = \Theta(n)$, com constante entre 1 e 2.

Observação (abordagem ótima em constante — torneio): uma análise mais fina do problema mostra que é possível encontrar o maior e o segundo maior usando exatamente $n + \lceil \log_2 n \rceil - 2$ comparações pelo método de torneio (eliminação por pares + reaproveitamento dos perdedores que enfrentaram o vencedor). Ainda assim, a complexidade assintótica permanece $\Theta(n)$.

Conclusão: o algoritmo apresentado é **ótimo em ordem assintótica**, com $T(n) = \Theta(n)$.
